Tema 2.3:

Resumen automático extractivo

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo
© Prof. David Candal Ventureira - 2026

Tabla de contenidos

1 Introducción

2 Graph of words (GoW)

3 TextRank (palabras clave)

4 TextRank (oraciones)

5 Implementación

6 Resumen multi-documento (MDS)

7 Evaluación

1

Introducción

Introducción: Resumen automático

- Leer manualmente un documento íntegro para extraer su información es un proceso lento y no escalable.
  - Legal: Auditorías de miles de contratos corporativos.
  - Finanzas: Procesamiento de cientos de informes de diferentes empresas antes de que abra el mercado.
  - Salud: Revisión de historiales clínicos desestructurados con décadas de evolución.

- Procesar textos largos a través de LLMs también tiene un alto coste:
  - El **coste computacional** de un LLM es $O(N^2)$. Los textos largos disparan el consumo de VRAM y la latencia TTFT (*time to first token*, fase de *prefill*).
  - **Problema "*Lost in the Middle*"**: Estudios prueban que los LLMs sufren una curva en "U" de atención. El modelo recupera bien el principio y el final del *prompt*, pero es ciego a la información central.



Introducción: Resumen automático

- Objetivo: Extraer matemáticamente las ideas clave de manera rápida, barata y sin alucinaciones.
- Tipos:
  - **Resumen extractivo**:
    - Un algoritmo matemático puntúa y extrae las oraciones exactas más importantes del texto original.
    - “Copy-paste” inteligente.
    - Ventajas: Coste computacional bajo, latencia baja y factualidad absoluta (son frases existentes en el documento)
    - Problemas: Textos estilo "Frankenstein“ (no conexos) y referencias anafóricas rotas (ej. Se extrae "Él firmó el contrato.", pero al aislar la frase no sabemos quién es "Él").



Introducción: Resumen automático

- Objetivo: Extraer matemáticamente las ideas clave de manera rápida, barata y sin alucinaciones.
- Tipos:
  - **Resumen abstractivo**:
    - Procesamos el texto con un modelo generativo para entender el texto y generar oraciones que resuman el documento (no contenidas en el documento).
    - Ventajas: Narrativa fluida, cohesiva y capaz de conectar ideas complejas con lenguaje natural.
    - Problemas: Computacionalmente caro, latencia alta y propenso a alucinaciones (inventa datos, que pueden generar situaciones muy problemáticas en sectores críticos).



Introducción: Resumen automático

- Ejemplos de resumen extractivo y resumen abstractivo:

El 17 de noviembre, la junta directiva de OpenAI anunció la destitución de su CEO, Sam Altman. La causa oficial fueron las fuertes discrepancias internas sobre los protocolos de seguridad.

Como respuesta inmediata, más de 500 empleados amenazaron con renunciar en bloque. Esto provocó la paralización total de las operaciones de la empresa.

- **Resumen extractivo**:
  - El 17 de noviembre, la junta directiva de OpenAI anunció la destitución de su CEO, Sam Altman.
  - Como respuesta inmediata, más de 500 empleados amenazaron con renunciar en bloque.

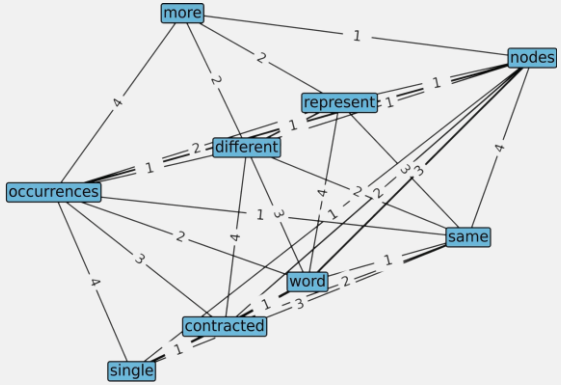
- **Resumen abstractivo**:
  - OpenAI **despidió** a Sam Altman por **desacuerdos** de seguridad, lo que desencadenó una **huelga masiva** que **detuvo** la **compañía**.

2

Graph of words (GoW)

GoW: Introducción

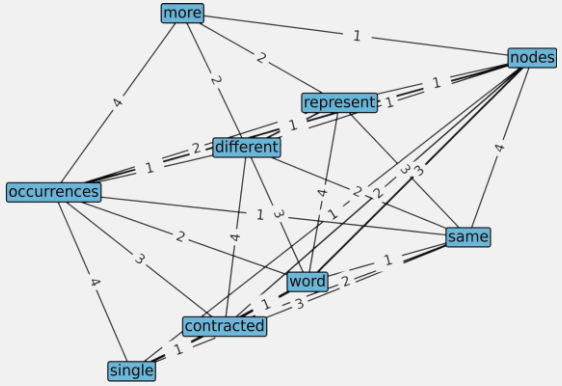
- Las palabras de un documento **no aportan información de manera secuencial y lineal**.
- Los conocimientos surgen a lo largo del documento, y se referencian más adelante (ciclos).
- Métricas de análisis:
  - **Frecuencia** (BoW): Miden la cantidad de veces que un término se repite a lo largo del documento.
    - Problema: Un término puede ser etiquetado como relevante simplemente porque se repite en una sección aislada.
  - **Centralidad** (GoW): Mide la relevancia de un término en base a cómo conecta con el resto de términos a lo largo de todo el documento.
- Para resumir un documento, nos interesa detectar cuáles son las palabras más relevantes a lo largo de todo el texto.



GoW: Introducción

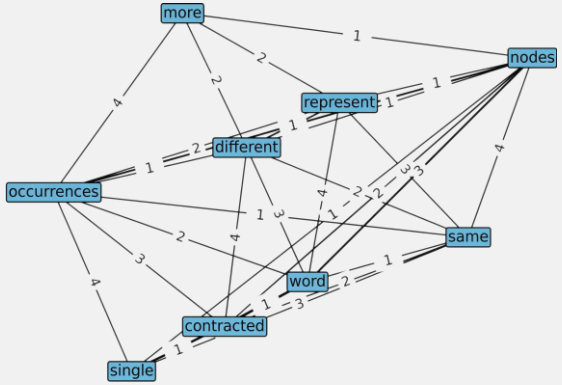
- En sistemas de resumen extractivo, la información de un documento debe modelarse como un **grafo** $G = (V, E)$:
  - **Vértices (V)**: Unidades de información (palabras únicas del texto).
  - **Aristas (E)**: Relación semántica entre los vértices (ej: co-ocurrencia de las palabras).

- En general, los grafos GoW se caracterizan por ser:
  - **Ponderados**: El peso de las aristas depende de la relación semántica entre los términos.
  - **No dirigidos**: Las aristas miden únicamente la relación semántica entre dos términos, no su orden de ocurrencia ni su grado de implicación.



GoW: Preparación del texto

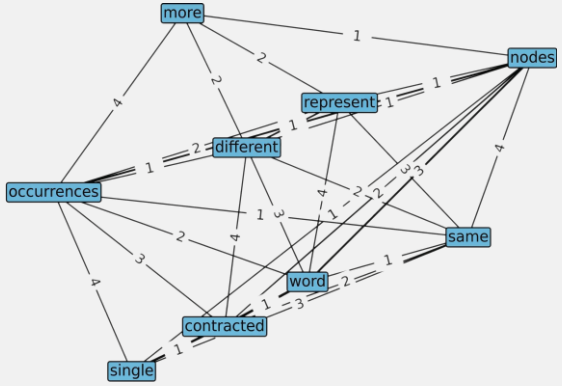
- Para poder generar grafos limpios, debemos normalizar previamente el texto:
  - **Tokenización**.
  - **Eliminación de stopwords**.
  - **Filtrado PoS (part-of-speech)**: Conservar únicamente los términos con mayor carga semántica (ej: sustantivos, adjetivos y verbos).
  - **Lematización**: Unificar variaciones morfológicas de los términos. Evita que un término se fragmente en el grafo, diseminando su peso en el texto.



- Ejemplo: “El nuevo algoritmo generó imágenes realistas instantáneamente”
  - -> [nuevo] [algoritmo] [generar] [imagen] [realista]

GoW: Generación del grafo

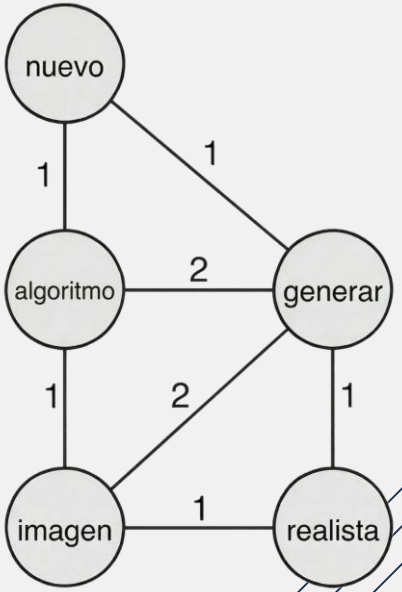
- Tras extraer los *tokens* del texto, utilizamos una ventana deslizante de tamaño N para detectar co-ocurrencia de términos y construir las aristas del grafo GoW:
  - **N pequeño (ej: 2-4)**: Captura únicamente relaciones muy locales.
  - **N grande (ej: 10-20)**: Captura relaciones más distantes, pero añade ruido al grafo, al abarcar más palabras.
- Ejemplo: [nuevo] [algoritmo] [generar] [imagen] [realista], con N = 3:
  - V1: [nuevo, algoritmo, generar]. Aristas: nuevo – algoritmo, algoritmo – generar y nuevo – generar.
  - V2: [algoritmo, generar, imagen]. Aristas: algoritmo – generar, generar – imagen y algoritmo – imagen.
  - V3: [generar, imagen, realista]. Aristas: generar – imagen, imagen – realista y generar – realista.



GoW: Generación del grafo

- Ejemplo: [nuevo] [algoritmo] [generar] [imagen] [realista], con N = 3:
  - V1: [nuevo] [algoritmo] [generar]. Aristas: nuevo – algoritmo, algoritmo – generar y nuevo – generar.
  - V2: [algoritmo] [generar] [imagen]. Aristas: algoritmo – generar, generar – imagen y algoritmo – imagen.
  - V3: [generar] [imagen] [realista]. Aristas: generar – imagen, imagen – realista y generar – realista.

- Resultado (aristas):
  - nuevo - algoritmo (peso 1).
  - **algoritmo - generar (peso 2)**.
  - nuevo - generar (peso 1).
  - **generar - imagen (peso 2)**.
  - algoritmo - imagen (peso 1).
  - imagen - realista (peso 1).
  - generar - realista (peso 1).



3

TextRank (palabras clave)

TR(pc): PageRank

- PageRank es un algoritmo diseñado para motores de búsqueda web.
- Se basa en la premisa de que una web es importante si otras webs importantes la enlazan.

$$S(Vi) = (1 - d) + d \sum_{V_j \in In(Vi)} \frac{S(Vj)}{|Out(Vj)|}$$

- Donde:
  - $S(Vj)$: Prestigio del nodo j.
  - $|Out(Vj)|$: Número de enlaces salientes desde el nodo j (se penaliza a los nodos que "votan" a todo el mundo).
  - d (factor de amortiguación): Probabilidad de seguir navegando a través de los enlaces de las webs que va visitando (ej: 0.85). Evita que el algoritmo se quede encerrado en bucles.



TR(pc): TextRank

- **TextRank** es un algoritmo basado en PageRank que busca **ordenar por relevancia semántica el texto dentro de un documento** (palabras clave de un documento).
- La formula de **relevancia de la palabra (WS)** es:

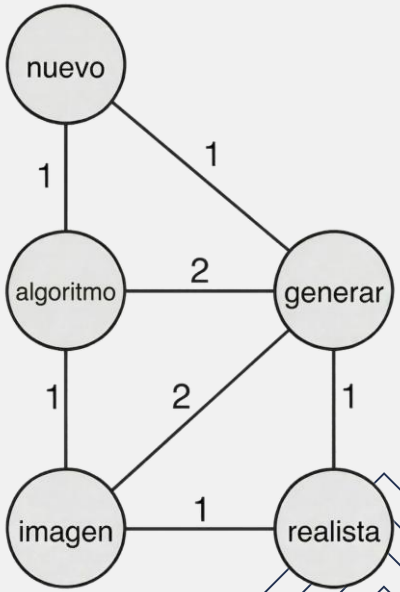
$$WS(Vi) = (1-d) + d \sum_{V_j \in In(Vi)} \frac{w_{ji}}{\sum_{V_k \in Out(Vj)} w_{jk}} WS(Vj)$$

- Donde:
  - $WS(Vj)$: Relevancia del nodo j.
  - w_{ji} : Peso de la arista entre i y j (co-ocurrencia).
  - $\sum_{V_k \in Out(Vj)} w_{jk}$: Sumatorio de los pesos de las aristas que salen de j hacia otros nodos (se penaliza la contribución de términos que co-ocurren frecuentemente con otros).



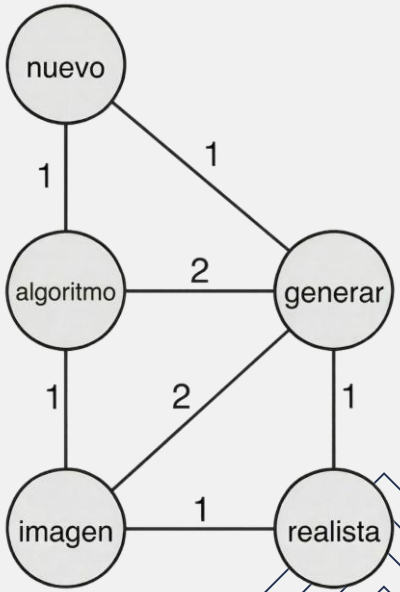
TR(pc): TextRank, ejemplo

- Ejemplo (d = 0.85):
  - Estado inicial (T_0):
    - WS(nuevo) = WS(algoritmo) = WS(generar) = WS(imagen) = WS(realista) = 1.0.
  - Iteración 1 (T_1):
    - WS(nuevo) = $(1-d)+d\left(\frac{1}{4}1+\frac{1}{6}1\right)=0.5042$
    - WS(algoritmo) = $(1-d)+d\left(\frac{1}{2}1+\frac{2}{6}1+\frac{1}{4}1\right)=1.0708$
    - WS(generar) = $(1-d)+d\left(\frac{1}{2}1+\frac{2}{4}1+\frac{2}{4}1+\frac{1}{2}1\right)=1.8500$
    - WS(imagen) = $(1-d)+d\left(\frac{1}{4}1+\frac{2}{6}1+\frac{1}{2}1\right)=1.0708$
    - WS(realista) = $(1-d)+d\left(\frac{1}{6}1+\frac{1}{4}1\right)=0.5042$



TR(pc): TextRank, ejemplo

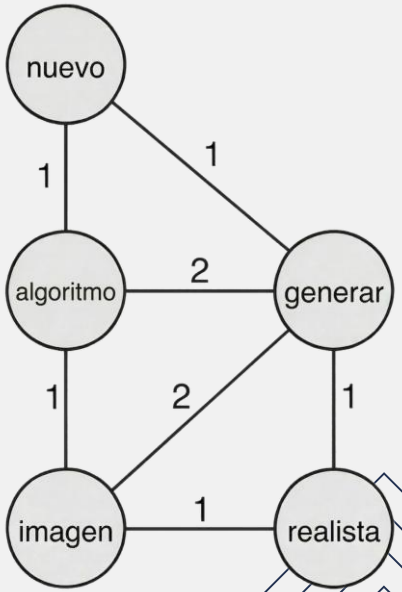
- Ejemplo (d = 0.85):
  - Iteración 2 (T_2):
    - WS(nuevo) = $(1-d)+d\left(\frac{1}{4}1.0708+\frac{1}{6}1.8500\right)=0.6396$
    - WS(algoritmo) = $(1-d)+d\,(\frac{1}{2}0.5042+\frac{2}{6}1.8500+\frac{1}{4}1.0708)=1.1160$
    - WS(generar) = $(1-d)+d\,(\frac{1}{2}0.5042+\frac{2}{4}1.0708+\frac{2}{4}1.0708+\frac{1}{2}0.5042)=1.4887$
    - WS(imagen) = $(1-d)+d\,(\frac{1}{4}1.0708+\frac{2}{6}1.8500+\frac{1}{2}0.5042)=1.1160$
    - WS(realista) = $(1-d)+d\left(\frac{1}{6}1.8500+\frac{1}{4}1.0708\right)=0.6396$



TR(pc): TextRank, ejemplo

- Ejemplo (d = 0.85):
  - Iteración 13 (T_{13}) (err < 10^{-4}):

Iteración	nuevo	algoritmo	generar	imagen	realista
1	0.5042	1.0708	1.8500	1.0708	0.5042
2	0.6396	1.1160	1.4887	1.1160	0.6396
3	0.5981	1.0808	1.6423	1.0808	0.5981
4	0.6123	1.0992	1.5770	1.0992	0.6123
5	0.6070	1.0906	1.6048	1.0906	0.6070
6	0.6091	1.0944	1.5930	1.0944	0.6091
7	0.6082	1.0928	1.5980	1.0928	0.6082
8	0.6086	1.0935	1.5959	1.0935	0.6086
9	0.6084	1.0932	1.5968	1.0932	0.6084
10	0.6085	1.0933	1.5964	1.0933	0.6085
11	0.6085	1.0932	1.5965	1.0932	0.6085
12	0.6085	1.0933	1.5965	1.0933	0.6085
13	0.6085	1.0933	1.5965	1.0933	0.6085



4

TextRank (oraciones)

TR(o): TextRank para extracción de oraciones

- Cambio de paradigma: Buscamos **extraer las N oraciones que sean más similares** al resto de oraciones del documento.
- Para pasar de extraer las palabras clave del documento a las oraciones más relevantes, necesitamos adaptar el algoritmo inicial:
  - **Vértices (V)**: Los nodos pasan de ser palabras a ser oraciones completas.
  - **Aristas (E)**: Toman como peso la similitud semántica entre dos oraciones.
- El resultado es un **grafo completo no dirigido** (todas los vértices se encuentran conectados entre si por una arista) que conecta las oraciones de un documento.
  - No utilizamos una ventana deslizante para determinar las aristas entre los nodos, conectamos cada par de oraciones sin importar la distancia entre ellas en el documento.

TR(o): Similitud de oraciones

- Cambio de paradigma: Buscamos extraer las N oraciones que sean más similares al resto de oraciones del documento.
- Para pasar de extraer las palabras clave del documento a las oraciones más relevantes, necesitamos **adaptar la función de similitud en el algoritmo inicial**:

$$Sim(s_i, s_j) = \frac{|\{w_k \mid wk \in s_i \cap wk \in s_j\}|}{log(|s_i|) + log(|s_j|)}$$

  - Donde:
    - $|\{wk \mid wk \in si \cap wk \in sj\}|$: Número de palabras contenidas tanto en s_i como en s_j.
    - $log(|si|) + log(|sj|)$: Suma de las longitudes de s_i y s_j. El denominador evita que oraciones muy largas dominen el grafo por estadística, pero la normalización logarítmica evita penalizarlas tanto como para que el resumen sólo contenga oraciones cortas.

TR(o): Similitud de oraciones

- El rendimiento de TextRank a la hora de extraer las oraciones más relevantes de un documento está acotado por:
  - La similitud semántica de dos oraciones se calcula exclusivamente a través del ratio de palabras que comparten, y **no por la similitud semántica** real.
    - Dos oraciones equivalentes que no compartan las mismas palabras obtendrán una similitud semántica igual a 0.
      - Ej: “Este coche es veloz” vs “Ese automóvil es rápido”.
  - Los sistemas de TextRank para oraciones son muy **sensibles a *stopwords***. Las oraciones que contengan palabras “vacías de significado” generarán aristas con otras oraciones mal filtradas, a pesar de que no compartan significado.
- Solución -> **TextRank basado en embeddings**: Calcular los *embeddings* de cada oración y calcular la similitud de cada par a través de la similitud coseno. Esto resuelve los problemas de estos sistemas con sinónimos y polisemia.

TR(o): Problemas de TextRank y *embeddings*

- Los sistemas TextRank para extracción de oraciones tienen **problemas de escalabilidad**:
  - Los grafos que se generan tienen N nodos y $\frac{N(N-1)}{2}$ aristas.
    - Para 100 oraciones, se generan 4950 aristas.
    - Para 1000 oraciones, se generan 499500 aristas.
  - Tanto calcular la similitud coseno de cada par de nodos como mantener esos resultados en memoria se vuelve inviable rápidamente a partir de un cierto valor de N.
- Adicionalmente, la similitud coseno de dos *embeddings* diferentes no es 0.0, si no que **siempre genera un valor residual**. Dado que todos los nodos están conectados con todos los demás, estas conexiones espurias pueden alterar artificialmente el valor de centralidad de las oraciones del documento.

TR(o): Dispersión

- Para solucionar los problemas de memoria y ruido semántico, aplicamos un proceso llamado ***dispersión*** (*sparsification*), que transforma el grafo completo en una matriz dispersa.
- Existen dos enfoques principales en industria:
  1. **Umbral de similitud absoluto**:
     - Definimos un umbral mínimo de similitud *t* (ej: *t*=0.7).
     - Para cada par de oraciones en los que la similitud sea menor a *t*, se elimina la arista que las una (se le asigna un peso 0).
     - La matriz de adyacencia resultante (que almacena los pesos obtenidos a través de la función de similitud entre cada par de oraciones) es:

$$W_{ij} = \begin{cases} sim(s_i, sj) & si\ sim(s_i, s_j) \geq t \\ 0 & en\ otro\ caso \end{cases}$$

TR(o): Dispersión

- Para solucionar los problemas de memoria y ruido semántico, aplicamos un proceso llamado ***dispersión*** (*sparsification*), que transforma el grafo completo en una matriz dispersa.
- Existen dos enfoques principales en industria:
  2. **Grafo de k-vecinos más cercanos (grafo k-NN)**:
     - Cada oración mantiene las aristas hacia las *k* oraciones con las que mantiene la similitud más alta.
     - Dada una oración s_i que tiene una oración s_j dentro de sus *k* oraciones más similares, puede darse el caso de que s_j no tenga a la oración s_j dentro de sus *k* oraciones más similares:
       - Opción grafo no dirigido: Cada oración añade aristas no dirigidas hacia las *k* oraciones más similares. Como resultado, las oraciones más centrales tendrán más de *k* aristas.
       - Opción grafo dirigido: Las *k* oraciones más similares generan aristas salientes desde la oración evaluada.

TR(o): Dispersión

- Resultado de aplicar dispersión:
  - La “relevancia” no fluye a través de enlaces que no tienen una similitud significativa. Previene que oraciones muy relevantes traspasen su relevancia hacia oraciones con las que no comparten similitud.
    - Se reducen significativamente los enlaces en el grafo, **reduciendo la complejidad de cada iteración**.
    - Al eliminar los enlaces que introducen ruido, **el algoritmo converge mucho más rápido**.
  - La matriz de adyacencia se vuelve dispersa:
    - Para cada oración, **se almacenan únicamente los pesos de las aristas** hacia otras oraciones similares.
    - Pasamos de *arrays* densos estándar a estructuras como CSR (*compressed sparse row*) en SciPy.
  - Nota: Sigue siendo necesario calcular la similitud coseno entre todos los pares de oraciones del documento.

TR(o): Problemas de la dispersión

- Al aplicar dispersión (por umbral de similitud o grafo k-NN), el grafo original puede **segmentarse en "islas" (componentes conexas)** de oraciones que tratan de sub/temas muy específicos.
- Si el grafo se fragmenta en múltiples componentes conexas, el resultado de aplicar TextRank sobre el documento sería equivalente a ejecutarlo de manera independiente en cada una de las componentes, y agregar los resultados obtenidos antes de clasificar las oraciones por relevancia.
- Problemas:
  - Islas de temáticas poco relevantes pueden dar lugar a oraciones de alta relevancia, que se cuelan entre los resultados finales del documento. Ej:
    - Componente principal: [$s_{1,1}$=0.4, $s_{1,2}$=0.35, $s_{1,3}$=0.3, $s_{1,4}$=0.2, $s_{1,5}$=0,15…].
    - Componente secundaria: [$s_{2,1}$=0.3, $s_{2,2}$=0.25, $s_{2,3}$=0.2, $s_{2,4}$=0.1…].
    - Oraciones relevantes (top 5): [$s_{1,1}$, $s_{1,2}$, $s_{1,3}$, $s_{2,1}$, $s_{2,2}$].
  - Adicionalmente, si el grafo es dirigido, pueden existir componentes con enlaces dirigidos que **sólo reciban relevancia** de otras componentes.

TR(o): Teleportación

- Para solucionar el problema de la fragmentación, introducimos la **teletransportación** a través de un **factor de amortiguación** (equivalente al de PageRank).
- El factor de amortiguación *d* se utiliza para establecer una probabilidad *1-d* de que una oración aleatoria del documento transfiera relevancia a una oración.
- Suponiendo que el valor de relevancia de las oraciones se inicializa a 1/N, la relevancia se puede calcular en cada iteración como:

$$v_{t+1} = d \cdot (W \cdot v_t) + \frac{(1-d)}{N}$$

- Donde:
  - v_t es el vector de relevancias de las oraciones en la iteración *t*.
  - *d* es el factor de amortiguación.
  - *W* es la matriz dispersa de pesos de las aristas del grafo.
  - *N* es el número de oraciones en el documento.

TR(o): Teleportación

- Resultados de la teletransportación:
  - La teletransportación genera un **grafo conexo**, que evita que la relevancia quede acotada a componentes conexas aisladas, garantizando que el ranking global converja adecuadamente.
  - Al simplificar la contribución de los “saltos aleatorios”, podemos obtener el comportamiento matemático que teníamos con un grafo completo operando con la matriz dispersa, que sólo almacena los pesos de las aristas con similitud significativa. Por tanto, se **mantiene el bajo coste de memoria y cómputo**.
  - Aunque estas “aristas virtuales” entre oraciones producen una transmisión ruidosa de relevancia, **el ruido introducido es uniforme**. Todas las oraciones transfieren la misma fracción de ruido. Al afectar a todas las oraciones por igual, el algoritmo converge adecuadamente, sin alterar los resultados finales de clasificación.

TR(o): Resumen de documentos extensos

- Calcular la similitud coseno entre cada par de oraciones del documento es un problema $O(N^2)$.
- Para documentos de tamaño medio (ej: < 50.000 oraciones) es posible que el *hardware* nos permita paralelizar el cómputo de la similitud a través de operaciones matriciales, haciendo que la consulta de similitud de todos los pares de oraciones sea más eficiente.
- **Para documentos inmensos** (ej: > 50.000 oraciones), es más conveniente **indexar los *embeddings* de las oraciones** para utilizar algoritmos de búsqueda semántica de tipo ANN (ej: HNSW).
  - La complejidad de insertar las oraciones en el índice es $O(N \log(N))$.
  - La complejidad de buscar los vecinos de las N oraciones en el índice de tamaño N es $O(N \log(N))$.

TR(o): Resumen de documentos extensos

- Aunque los algoritmos ANN nos ayudan a trabajar con documentos extensos, generalmente los documentos extensos incluyen **diferentes secciones/capítulos**.
- A través del algoritmo que hemos visto hasta ahora, comparamos todas las oraciones. Si cada sección trata temáticas independientes, **la similitud entre oraciones de diferentes secciones será baja** (**a pesar de ser relevantes**).
- En general, nos interesará **obtener las oraciones más relevantes de cada sección** y, a partir de estas oraciones, determinar las más relevantes para el documento.
- Procedimiento:
    1. Dividimos el documento en secciones secuenciales (por sección o por tamaño).
    2. Ejecutamos TextRank en cada sección, obteniendo r' oraciones relevantes para cada una.
    3. Combinamos estas oraciones para ejecutar TextRank, obteniendo las r oraciones más relevantes del documento.

5

Implementación

Implementación: Introducción

- Hoy en día, la industria no usa el TextRank original (2004), sino que aplica las técnicas que hemos visto hasta el momento:
  1. **Similitud semántica**: Calculamos los *embeddings* de cada oración del documento.
  2. **Optimización de la topología**: Calculamos la similitud entre cada par de oraciones, aplicando un umbral semántico o k-NN para podar las aristas espurias. Tras ello, aplicamos el factor de amortiguación (teletransportación).
  3. **Se ejecuta TextRank** sobre el grafo "conectado" generado, obteniendo un ranking de oraciones relevantes.
     a) Si el documento es extenso y la temática es la misma a lo largo del mismo, usamos **ANN** en lugar de calcular la similitud coseno de cada par de *embeddings*.
     b) Si el documento tiene secciones independientes, aplicamos **TextRank jerárquico** (aplicamos TextRank a cada sección y al resultado obtenido de ellas).
  4. Utilizamos **MMR (*maximal marginal relevance*)** para que el resultado sea diverso.
  5. **Ordenamos las oraciones** en base al orden en el que aparecen en el texto.

Implementación: Introducción

- A nivel de implementación, un sistema de resumen extractivo requiere:
  1. **Segmentación de oraciones y vectorización**: Usamos tokenizadores estadísticos (NLTK, spaCy) para gestionar abreviaturas y decimales. Tras ello, convertimos las oraciones en *embeddings*.
  2. **Construcción de la topología**: Construir el grafo disperso (grafo completo -> dispersión -> teletransporte).
  3. **Ranking**: Ejecutamos el algoritmo de TextRank sobre el grafo. Obtenemos un diccionario con la relevancia de cada oración.
  4. **Selección (relevancia y diversidad):** Aplicamos algoritmos de reranking como MMR para penalizar la repetición de información y maximizar la cobertura.
  5. **Reordenado cronológico**: Las oraciones seleccionadas se ordenan por su índice de aparición original en el texto, en lugar de por su relevancia, para preservar la cohesión narrativa.

Implementación: Segmentación y vectorización

- **Paso 1: Segmentación y vectorización**.
  - **Segmentación**: Necesitamos modelos específicos para segmentar oraciones, que nos permitan desambiguar los límites de cada oración.
  - Ej:
    - "El Dr. Smith llegó a EE.UU. ayer tras un vuelo de $1.500.“
    - Dividir por puntos generaría 5 oraciones sin sentido
    - Ejemplos estándar: nltk.sent_tokenize o spaCy.

  - **Vectorización**: Una vez obtenida la lista de oraciones, usamos un modelos de *sentence embeddings* (SBERT) para convertirlas en *embeddings* densos, que nos permitan comparar su similitud.

Implementación: Segmentación y vectorización

- **Paso 1: Segmentación y vectorización.**

```python
import nltk
import spacy
from pysbd import Segmenter
#nltk.download('punkt_tab')

texto = '''El Dr. Smith viajó a los EE.UU. el año pasado.
Allí compró un equipo por 3.500 euros. ¡Fue una gran inversión!'''

# --- OPCIÓN 1: NLTK ---
# Ligero, precisión media
nltk.download('punkt', quiet=True)
frases_nltk = nltk.sent_tokenize(texto, language='spanish')
print(f"NLTK ({len(frases_nltk)} frases):", frases_nltk)
```

```python
# --- OPCIÓN 2: spaCy ---
# Lento, muy preciso
# python -m spacy download es_core_news_sm
nlp = spacy.load("es_core_news_sm")
doc = nlp(texto)
frases_spacy = [sent.text for sent in doc.sents]
print(f"spaCy ({len(frases_spacy)} frases):", frases_spacy)

# --- OPCIÓN 3: PySBD ---
# Preciso y rápido
segmenter = Segmenter(language='es', clean=False)
frases_pysbd = segmenter.segment(texto)
print(f"PySBD ({len(frases_pysbd)} frases):", frases_pysbd)
```

Implementación: Construcción de la topología y ranking

- **Pasos 2 y 3: Construcción de la topología y ranking**.
  - **Construcción de la topología**:
    1. Calculamos la similitud entre cada par de oraciones.
    2. Aplicamos dispersión:
       - Umbral: Eliminamos las aristas con similitud baja (ej: <0.7).
       - K-NN: Obtenemos los k nodos más cercanos a cada oración.
    3. Teletransportación: Inyectamos el factor de amortiguación (ej: d = 0.85) para garantizar la convergencia.
  - **Ranking**:
    - Ejecutamos el algoritmo de TextRank:
      1. Iterativamente, transferimos relevancia desde cada nodo al resto de nodos en base a la matriz de adyacencia.
      2. Repetimos el proceso hasta que el error medio entre iteraciones sea menor a un umbral preestablecido.
    - La salida de este algoritmo es un diccionario, en le que las claves son las oraciones del documento y sus valores son su valor de relevancia tras aplicar TextRank.

Implementación: Construcción de la topología y ranking

- **Pasos 2 y 3: Construcción de la topología y ranking.**

```python
import numpy as np
import networkx as nx
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

import logging
logging.getLogger("transformers").setLevel(logging.ERROR)
logging.getLogger("sentence_transformers").setLevel(logging.ERROR)

# Documento de ejemplo
oraciones = [
    # Cardiología e IA
    "La inteligencia artificial permite detectar enfermedades cardíacas analizando electrocardiogramas con gran precisión.",
    "El uso de modelos de IA en cardiología facilita la identificación de patologías del corazón mediante el análisis de ECG.",
    "Los sistemas computacionales actuales son capaces de encontrar anomalías en el ritmo cardíaco de forma automatizada.",
    # Telemedicina y Acceso Rural
    "La telemedicina ha permitido que pacientes en zonas rurales accedan a especialistas sin necesidad de desplazarse.",
    # Ética y privacidad
    "La privacidad de los datos médicos sigue siendo el mayor desafío ético para la implementación de la salud digital.",
    # General
    "La tecnología moderna está cambiando radicalmente la forma en la que entendemos la asistencia sanitaria actual."
]
```

Implementación: Construcción de la topología y ranking

- **Pasos 2 y 3: Construcción de la topología y ranking.**

```python
# Umbrales
umbral_semantico = 0.70
factor_amortiguacion = 0.85

# Debug
print(f"--- Documento de {len(oraciones)} oraciones: ---")
for i, ot in enumerate(oraciones):
    print(f"[{i}]: {ot}")
print("-" * 30)


# 1. Vectorización
print("1. Generando embeddings...")
model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
embeddings = model.encode(oraciones)
```

```python
# 2. Topología
print(f"2. Construyendo el grafo disperso (Umbral > {umbral_semantico})...")

# Calculamos la matriz de similitud coseno completa
matriz_similitud = cosine_similarity(embeddings)

# Aplicamos la dispersión (umbral)
grafo = nx.Graph()  # Grafo no dirigido ponderado
n = len(oraciones)
aristas_creadas = 0
for i in range(n):
    for j in range(i + 1, n):  # Evitar duplicados
        similitud = matriz_similitud[i][j]
        if similitud > umbral_semantico: # Poda por umbral
            grafo.add_edge(i, j, weight=similitud)
            aristas_creadas += 1
print(f"Topología creada: {grafo.number_of_nodes()} nodos y {aristas_creadas} aristas.")
print(f"(De {int(n*(n-1)/2)} conexiones posibles, hemos podado {int(n*(n-1)/2) - aristas_creadas}).")
```

Implementación: Construcción de la topología y ranking

- **Pasos 2 y 3: Construcción de la topología y ranking.**

```
# Ranking
print(f"3. Ejecutando PageRank ponderado con teleportación (d={factor_amortiguacion})...")

# nx.pagerank aplica PageRank, usando los pesos del grafo y el factor de amortiguación
scores = nx.pagerank(grafo, alpha=factor_amortiguacion, weight='weight')

# Resultado
# Ordenamos el diccionario por score para ver los ganadores
ranking_ordenado = sorted(scores.items(), key=lambda item: item[1], reverse=True)

print("-" * 30 + "OUTPUT (Diccionario de relevancia):")
for id_oracion, score in ranking_ordenado:
    print(f"  Frase [{id_oracion}] (Score: {score:.4f}): {oraciones[id_oracion]}")

print("-" * 30 + f"LAS 3 MÁS RELEVANTES SON: Frase [{ranking_ordenado[0][0]}], Frase [{ranking_ordenado[1][0]}] y Frase [{ranking_ordenado[2][0]}]")
```

Implementación: Construcción de la topología y ranking

- **Pasos 2 y 3: Construcción de la topología y ranking.**

```
--- Documento de 6 oraciones: ---
[0]: La inteligencia artificial permite detectar enfermedades cardíacas analizando electrocardiogramas con gran precisión.
[1]: El uso de modelos de IA en cardiología facilita la identificación de patologías del corazón mediante el análisis de ECG.
[2]: Los sistemas computacionales actuales son capaces de encontrar anomalías en el ritmo cardíaco de forma automatizada.
[3]: La telemedicina ha permitido que pacientes en zonas rurales accedan a especialistas sin necesidad de desplazarse.
[4]: La privacidad de los datos médicos sigue siendo el mayor desafío ético para la implementación de la salud digital.
[5]: La tecnología moderna está cambiando radicalmente la forma en la que entendemos la asistencia sanitaria actual.
------------------------------
1. Generando embeddings...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faste
r downloads.
Loading weights: 100%|                                                                                | 199/199 [00:00<00:00, 1813.48it/s]
2. Construyendo el grafo disperso (Umbral > 0.7)...
Topología creada: 3 nodos y 2 aristas.
(De 15 conexiones posibles, hemos podado 13).
3. Ejecutando PageRank ponderado con teleportación (d=0.85)...
------------------------------OUTPUT (Diccionario de relevancia):
  Frase [0] (Score: 0.4865): La inteligencia artificial permite detectar enfermedades cardíacas analizando electrocardiograma
s con gran precisión.
  Frase [1] (Score: 0.2640): El uso de modelos de IA en cardiología facilita la identificación de patologías del corazón medi
ante el análisis de ECG.
  Frase [2] (Score: 0.2495): Los sistemas computacionales actuales son capaces de encontrar anomalías en el ritmo cardíaco de
 forma automatizada.
------------------------------LAS 3 MÁS RELEVANTES SON: Frase [0], Frase [1] y Frase [2]
```

Implementación: Selección

- **Paso 4: Selección**.
  - Los *k* primeros resultados del ranking pueden ser **demasiado similares**, lo que provoca que en conjunto no sean capaces de resumir el documento.
  - La información de un documento es submodular: **la primera oración aporta la mayor información** (100% de novedad). **Las siguientes oraciones añaden información complementaria** a las ya existentes.
  - Solución: **MMR (*maximal marginal relevance*)**: Añadimos iterativamente oraciones que complementen la información semántica de las ya extraídas.
    1. Extraemos la oración mas relevante.
    2. Obtenemos la siguiente oración más relevante, considerando tanto su puntuación en el ranking como su similitud a las oraciones ya extraídas.
    3. Repetimos el proceso hasta obtener las k oraciones objetivo.

$$MMR = \arg\max_{s \in R \setminus S} [(1 - \lambda) \cdot ranking - \lambda \cdot \max_{s' \in S} (sim_coseno(s, s'))]$$

  - λ representa la diversidad (0 no penaliza similitud, 1 penalización severa).

Implementación: Reordenado cronológico

- **Paso 5: Reordenado cronológico**.
  - Una vez seleccionadas las oraciones, debemos **ordenar el resultado según la ubicación de cada oración en el texto** (y no según su relevancia), para que el resultado a mostrar sea coherente.
  - Ejemplo:
    - Documento: Guión de *Buscando a Nemo*.
      - “Tras superar innumerables peligros, padre e hijo se reencuentran y regresan a casa”.
      - “Nemo, un pequeño pez payaso, es capturado por un buceador en la Gran Barrera de Coral”.
      - “Marlin emprende una aventura a través del océano para encontrar y rescatar a su hijo Nemo”.
  - Procedimiento:
    1. Almacenamos el índice de cada oración.
    2. Organizamos los resultados de ranking en base a su índice.

6

Resumen multi-documento

RM: Introducción

- Hasta ahora, hemos visto técnicas que permiten la extracción de oraciones relevantes de un documento a modo de resumen.
- El reto industrial actual no es extraer la información de un único documento, sino a través de un **conjunto de múltiples fuentes** (webs, PDFs, bases de datos…), con diferentes terminologías y tono, y potenciales contradicciones.
- El **resumen multi-documento (MDS, del inglés *multi-document summarization*)** es el proceso de tomar varios textos distintos que hablan de un **mismo tema** y **destilar una única versión coherente sin repetir información**:
  - **Compresión de contexto**: Maximizar la densidad de información del resumen del conjunto de documentos.
  - **Resolución de conflictos**: Gestionar discrepancias entre textos y anacronismos (información obsoleta).
  - **Personalización**: Sesgar el resultado hacia la intención de la búsqueda específica del usuario.

RM: El problema de las anáforas

- **El problema de las anáforas**: Al extraer una oración se pierden las referencias no explícitas a la información de su contexto, dando lugar a oraciones huérfanas.
- Ejemplo:
  - Oración 1: “La empresa OpenAI lanzó su nuevo modelo generativo ayer.”
  - Oración 2: “Su CEO afirmó que éste es su mayor logro.”
  - Si el algoritmo selecciona la oración 2, se rompen todas sus anáforas, al estar la oración fuera de su contexto.
- Para solucionar este problema, utilizamos **CDCR (*cross-document coreference resolution*)**:
  - Antes de construir el grafo, debemos ejecutar un preprocesamiento que **identifique y resuelva estas ambigüedades**.
  - **Sustituimos pronombres** (él, ella, esto, la compañía) por la **entidad nominal explícita** a la que hacen referencia dentro de su propio flujo de origen.
  - Esto garantiza que, independientemente de qué oraciones elija TextRank, **el resumen final siempre será autocontenido**.
  - Ejemplo: “El CEO [de OpenAI] [Sam Altman] afirmó que [GPT-5] es su mayor logro [de Sam Altman].”

RM: Construcción del grafo

- En MDS, construimos **una única matriz de adyacencia** que contiene todas las oraciones de todas las fuentes simultáneamente.
  - Resumir cada documento de manera independiente puede **eliminar información que, no siendo relevante de manera local, lo sea de manera global**. A través de un único grafo, el algoritmo de centralidad observa toda la red de relaciones a la vez. Una oración con baja relevancia local puede ganar una centralidad masiva si es el único puente que conecta dos subtemas presentes en diferentes documentos.
  - Adicionalmente, si la primera etapa de ranking falla, **el segundo nivel no tiene capacidad para recuperar la información perdida**.
  - Al enfrentar las oraciones directamente en el mismo espacio vectorial, **las contradicciones** (ej. datos distintos sobre un mismo hecho) **emergen en el grafo** como clústeres enfrentados, permitiendo una detección que el método jerárquico ignora.

RM: Construcción del grafo

- Construcción del grafo global:
  - Tras el preprocesamiento (CDCR), **unificamos todas las oraciones** de todos los documentos (D_1, D_2, …, D_n) en una única matriz de adyacencia.
  - Las oraciones de distintos documentos que reportan el mismo hecho **forman clústeres fuertemente conectados** (cliques).
  - TextRank fluirá naturalmente hacia estos cliques, seleccionando la oración de "consenso" que mejor representa la información compartida.
  - Problema: Si un documento es significativamente más largo que los demás, sus aristas intra-documento generarán una centralidad artificialmente alta. Para garantizar la representatividad de todas las fuentes, ajustamos la matriz de similitud para que **penalice las aristas intra-documento y premie las aristas inter-documento ($\boldsymbol{\beta} > \propto$)**:

$$W_{ij} = \begin{cases} \propto \cdot sim(s_i, sj) & si\ Doc(s_i) == Doc(s_j) \\ \beta \cdot sim(s_i, sj) & si\ Doc(s_i) \neq Doc(sj) \end{cases}$$

RM: Selección en MDS

- Cuando trabajamos con un único documento, es habitual que **los primeros resultados de TextRank sean oraciones semánticamente equivalentes**, especialmente si el documento es extenso.
- Cuando combinamos múltiples documentos, las oraciones con mayor relevancia suelen ser **versiones diferentes de una misma expresión**.
- Ejemplo:
  - Oración 1: "OpenAI lanzó GPT-5 ayer.“
  - Oración 2: "Ayer se produjo el lanzamiento de GPT-5 por OpenAI.“
  - Oración 3: "La compañía de Sam Altman presentó GPT-5 el lunes."
- En MDS, MMR pasa de ser una simple mejora a una **técnica necesaria** para poder obtener un resumen diverso del conjunto de documentos.
- Debemos sesgar el parámetro λ hacia valores que **fomenten la diversidad**. Así, MMR fomenta la extracción de oraciones que representen cada uno de los contenidos relevantes en el conjunto, en lugar de simplemente entregar diferentes versiones de la oración más relevante.

RM: Contradicciones

- **Contradicciones lógicas**: Dos fuentes pueden ser semánticamente muy similares pero decir lo opuesto.
  - Ejemplo:
    - Doc A: “Los ingresos subieron un 20%.”.
    - Doc B: “Los ingresos cayeron un 20%.”.
    - La similitud coseno será muy alta (ej: 0.9) y un grafo simple las fusionaría en un falso consenso.
  - Solución: Grafos signados y NLI (*natural language inference*): Utilizamos un modelo de inferencia para etiquetar las aristas. Si detecta contradicción, **el peso de la arista se vuelve negativo** (ej: -0.90).
  - Como resultado, **las oraciones tienden a no agregarse**, lo que permite que **ambas sean extraídas y analizadas**.

- **Contradicciones por obsolescencia**: La información en los documentos (especialmente en temáticas volátiles como la tecnología) tiene fecha de **caducidad**. Generalmente sólo nos interesa la información más actualizada.
  - Para penalizar documentos antiguos, aplicamos un **decaimiento exponencial**: Multiplicamos el peso de la arista del nodo por una función de antigüedad (t).
  - Ejemplo: $W_{temporal} = W_{original}\, e^{-\lambda(t_{actual} - t_{doc})}$
  - Resultado: La relevancia **fluye hacia las oraciones más recientes**, priorizando la actualización frente a información repetida antigua.

RM: Problemas de subsunción y ordenación

- **Problema de la subsunción**: En MDS, a menudo una oración genérica es semánticamente similar a una detallada, pero la detallada aporta más información.
  - Doc 1: "Apple anunció un nuevo teléfono.“.
  - Doc 2: "Apple anunció el iPhone 17 Pro Max con nuevo procesador.“
  - Para solucionar este problema, utilizamos un modelo de NLI para detectar si una oración implica a otra (A->B). En dicho caso, **creamos una arista dirigida desde la oración más genérica hacia la más detallada**, de manera que la relevancia fluye hacia la que aporta más información.
- **Problema de la ordenación**: Para ordenar las oraciones más relevantes del conjunto, podemos emplear:
  - **Organización por temática**: Se agrupan por similitud semántica.
  - **Referencia temporal del documento de origen**.
  - **Generación asistida por LLM**.

RM: MDS y recuperación de información

- TextRank distribuye de manera uniforme la relevancia inicial entre las oraciones. Esto genera un resumen generalista del documento.
- En sistemas vinculados a recuperación de información, no nos interesa lo más relevante de los documentos. **Buscamos lo más relevante para responder a una consulta del usuario**. Esto se conoce como **Query-Focused Summarization (QFS)**.
- **Personalized PageRank (PPR)**: Modificamos la fórmula de relevancia, tratando de **sesgarla hacia la consulta del usuario**. Para ello, modificamos el vector de salto aleatorio del algoritmo para emular que los teletransportes ocurran hacia oraciones similares semánticamente con la consulta del usuario.

$$v_{t+1} = d \cdot (W \cdot v_t) + (1 - d) \cdot u$$

- Donde u es un vector normalizado (la suma de sus elementos es 1) que toma como valores la **similitud entre cada oración y la consulta**:

$$u_i = sim(si, query)$$

7

Evaluación

Evaluación: Introducción

- A diferencia de otros problemas de ML (clasificación, regresión…), donde tenemos una etiqueta categórica o valor numérico, en el resumen de texto **no existe un "resumen perfecto"**.
- Ejemplo: Tres expertos humanos generarán tres resúmenes distintos, y todos pueden ser excelentes.
- Un resumen extractivo no tiene alucinaciones (se trata de oraciones que existen en el documento). La evaluación de estos sistemas se centra en analizar la **relevancia y cohesión** de las oraciones resultado.
- Para la evaluación de sistemas de resumen extractivo, **generamos manualmente resúmenes (referencias)** y comparamos matemáticamente el **resumen extractivo (candidato)** frente a ellas.
- Dimensiones críticas: **informatividad** (“recall” de la información en el documento/conjunto), **redundancia** y **utilidad** para responder a una consulta.

Evaluación: BERTScore

- Mecanismos de evaluación:
  - ROUGE (tradicional): Mide coincidencia de n-gramas. No captura semántica.
  - **BERTScore**: Compara los *embeddings* contextuales de las ***palabras*** de las **oraciones de las referencias frente a las del resumen candidato**.
    - Dos oraciones similares pueden tener *embeddings* similares, **pero no compartir información clave (detalles)**.
    - A través de los *embeddings* de palabra, **buscamos que todos los conceptos se encuentren en el resumen candidato**.
    - **Utilizamos las métricas clásicas de recuperación de información** (*recall*, precisión, F1-score…) para evaluar el grado en el que el candidato ha recogido las ideas del documento/conjunto.
    - En lugar de contar palabras, **usamos el máximo valor de similitud entre los términos del resumen candidato y las referencias**.

Evaluación: BERTScore

- Mecanismos de evaluación:
  - ROUGE (tradicional): Mide coincidencia de n-gramas. No captura semántica.
  - **BERTScore**: Compara los *embeddings* contextuales de las ***palabras*** de las **oraciones de las referencias frente a las del resumen candidato**.
    - Ejemplo:
      - Recall BERTscore: $R_{BERT} = \frac{1}{|Ref|}\sum_{x_i \in Ref} \max_{\hat{x}_j \in Cand} sim(xi, \hat{x}_j)$
      - Precisión BERTscore: $P_{BERT} = \frac{1}{|Cand|}\sum_{\hat{x}_j \in Cand} \max_{x_i \in Ref} sim(xi, \hat{x}_j)$
      - F1-score BERTscore: $F_{BERT} = 2\frac{P_{BERT} \cdot RBERT}{P_{BERT} + R_{BERT}}$
    - Donde:
      - *Cand* es el conjunto de los *embeddings* de palabra del resumen candidato.
      - *Ref* es el conjunto de los *embeddings* de palabra de los resúmenes de referencia.

Evaluación: Recuperación de información

- En los sistemas de resumen extractivo vinculados a recuperación de información, no nos interesan las oraciones más relevantes de los documentos. Nos interesan las **oraciones relevantes para la consulta**.
- Por ello, es necesario evaluar (generalmente **de manera manual**) el grado en el que la respuesta es útil para la consulta:
  - **Precisión de contexto**: Mide el ratio de oraciones en el resumen generado que son útiles para la consulta, frente al total de oraciones en el resumen.

$$P_{CTXT} = \frac{\sum_{x_i \in cand} util(xi,\ \ query)}{|Cand|}$$

  - ***Recall* de contexto**: Mide el ratio de oraciones útiles para la consulta presentes en el documento que han sido extraídas en el resumen.

$$R_{CTXT} = \frac{\sum_{x_i \in cand} \widehat{util}(xi,\ \ query)}{\sum_{x_j \in doc} \widehat{util}(xj,\ query)} \approx \frac{\sum_{x_i \in cand} \widehat{util}(xi,\ \ query)}{\sum_{\hat{x}_j \in ref} \widehat{util}(\hat{x}_j,\ \ query)}$$

- Donde:
  - $util$ mide el grado de utilidad de manera continua ($util \in [0,1]$) o categórica ($util \in \{0,1\}$), mientras que $\widehat{util}$ mide la utilidad de manera categórica ($\widehat{util} \in \{0,1\}$).

Evaluación: LLM-as-a-judge

- BERTscore evalúa la calidad de los conceptos (*tokens*) de las oraciones extraídas, pero **no es capaz de analizar la coherencia global de la respuesta ni su fluidez narrativa**.
- Podemos utilizar un LLM para consultar estas dos métricas en base a las oraciones extraídas (y la consulta del usuario).
- Dimensiones: Coherencia y veracidad (también relevancia/precisión de la respuesta frente a la consulta, si es el caso).
- Aplicar *LLM-as-a-judge* da lugar a los siguientes problemas:
  - **Sesgo de verbosidad**: El LLM tiende a puntuar mejor los resúmenes más largos.
  - **Sesgo de posición**: Si buscamos comparar dos resúmenes en un prompt, el LLM tenderá a preferir la primera opción.
  - **Sesgo de egocentrismo**: El LLM preferirá textos que imitan su propio estilo de redacción.
  - **Contaminación de datos**: Si evaluamos con *datasets* públicos, el LLM probablemente fue entrenado con los textos originales, lo que produce un sesgo hacia “su resumen ideal”, falseando las métricas.

¿Preguntas?